Why and How Jira Migrated from JSON to Protobuf

Source: Arpit Bhayani Dissection • Atlassian Engineering
Focus: Caching Tier Bottlenecks & Protobuf Binary Internals

1. The Context: Jira's Massive Caching Tier

Atlassian Jira's core entity—the Issue document—is a complex data structure containing standard metadata (keys, statuses, timestamps) and dozens of custom fields. To serve millions of developers viewing boards and sprint backlogs at high concurrency, Jira relies on a large-scale distributed caching tier (Memcached and Redis) holding hundreds of millions of serialized issue records.

Historically, Jira stored these documents as plain text **JSON strings**. As data volume expanded, JSON became an architectural bottleneck across CPU, memory footprint, and network bandwidth.

2. The Hidden Architectural Tax of JSON at Scale

```
Sample JSON Document (135 bytes):
{"issue_id":1098234,"project_key":"PROJ","status_id":3,"created_at":"2026-10-25T14:30:00Z"}

Overhead Breakdown:
- Key Repetition: "issue_id", "project_key", "status_id" repeated on every single row (60%+ wasted payload)
- Parsing Tax: Character scanning, quote escaping, UTF-8 validation, ASCII-to-integer conversion
- JVM Memory Churn: Generating millions of short-lived Java String allocations triggering severe GC pauses
- Cache Capacity Tax: Large payloads caused rapid Memcached memory saturation and high eviction rates
```

3. Protocol Buffers (Protobuf) Architecture

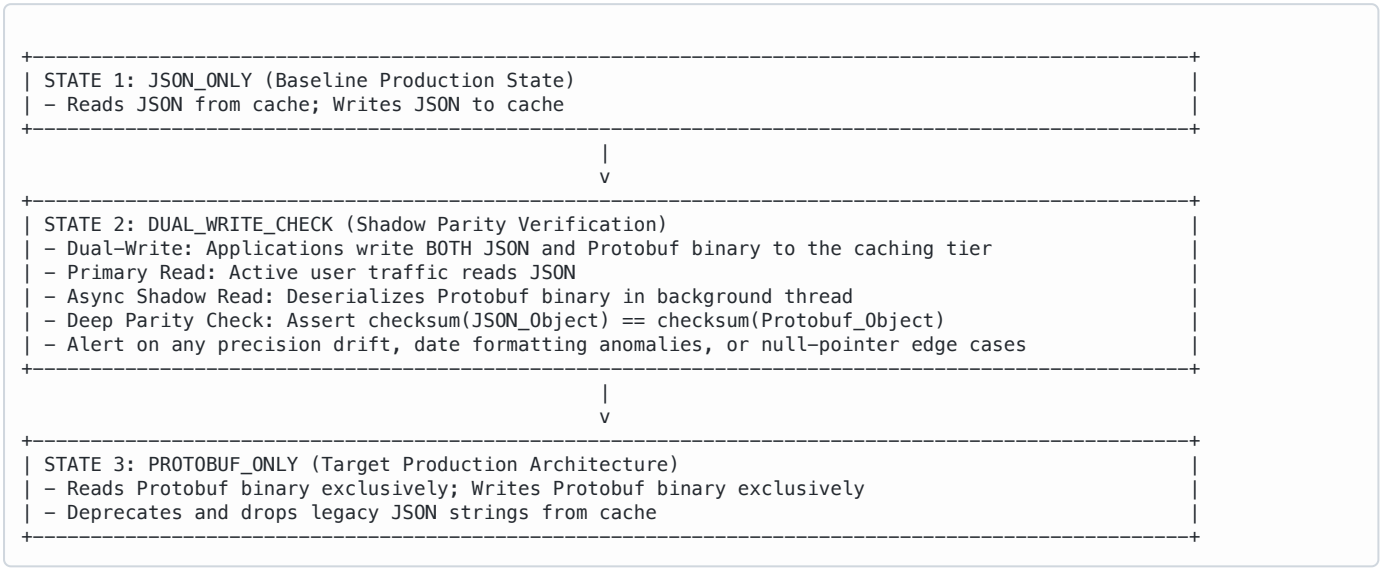
Google's Protocol Buffers is a binary serialization format that replaces human-readable text strings with compiled, strongly typed binary encodings. In Protobuf:

- **Compiled Schema:** Data structures are defined in `.proto` files and compiled ahead-of-time into optimized native code.
- **Tag-Length-Value (TLV) Wire Format:** Field names are replaced with compact 1-byte integer tags. The wire format contains zero field name strings or delimiters (`""`, `:`, `{}`).
- **Varints (Variable-Length Quantities):** Small integers (e.g. status ID = 3) are packed into a single byte instead of 4 or 8 bytes.

> **Binary Efficiency:** The same 135-byte JSON record compresses to just **18 bytes** in Protobuf binary format—an immediate **87% payload size reduction** with zero loss of fidelity.

4. Zero-Downtime Rollout: The 3-State Shadow Pattern

A migration across hundreds of millions of active cached records cannot afford serialization defects. Atlassian designed a strict **Three-State Feature Flag Migration Pipeline**:



5. Schema Evolution & Compatibility Rules

To prevent breaking downstream microservices as Jira entities evolve, Atlassian enforces strict Protobuf evolution rules:

- **Immutable Field Numbers:** Field tags ($1, 2, 3$) are permanent identities. Field names can be refactored, but numbers are never changed.
- **Purely Additive Updates:** New attributes are declared as optional or proto3 default fields. Older software ignores unrecognized tags without error.
- **Reserved Tags:** Deprecated fields are marked `reserved` to prevent future reuse of tag numbers.

Quantitative Results & Systems Engineering Lessons

Source: Arpit Bhayani Dissection • Atlassian Engineering
Focus: 55% Cost, 75% CPU, 33x Deserialization & Takeaways

6. Production Impact & Quantitative Results

The shift to Protobuf across Jira's caching layer delivered massive architectural improvements:

Performance / Cost Dimension	Legacy JSON Architecture	Protobuf Architecture	System Impact
Payload Data Size	$45 ext{ KB}$ per issue document	$9 ext{ KB}$ per issue document	**80% reduction in data size**
Deserialization Latency	$ pprox 1.2 ext{ ms}$ per issue parse	$ pprox 0.036 ext{ ms}$ ($36 ext{ }\mu ext{s}$)	**33x faster deserialization**
Caching Tier CPU Usage	High CPU parsing text strings	Direct binary memory mapping	**75% reduction in CPU consumption**
Infrastructure & Cloud Cost	100% baseline expenditure	45% cloud expenditure	**55% direct cloud cost reduction**
Cache Hit Ratio	91% (Frequent memory evictions)	98.5% ($5 imes$ more records in same RAM)	Relational database load dropped significantly

7. First-Principles System Design Takeaways

- **Serialization Format Governs System Boundaries:** JSON is ideal for public boundary APIs and browser debugging. However, for internal microservices, RPCs, and high-throughput caching, binary protocols are mandatory.
- **Compound Efficiencies of Compact Encoding:** An 80% drop in payload size produces compound benefits: reduced network packet fragmentation, $5 imes$ higher cache density in RAM, lower CPU cycles, and reduced JVM garbage collection pauses.
- **Shadow Verification Enables Fearless Refactoring:** Never attempt a big-bang data migration. The 3-state dual-write and asynchronous shadow comparison pattern enables zero-risk verification under real production workloads.
- **Enforced Type Safety Prevents Runtime Errors:** Strongly typed Protobuf schemas catch data validation and type mismatches at compile time rather than failing in production at runtime.